

Rational Basis Functions for Continuous-Kernel Graph Convolutions

Draft — technical description, training recipe, and two main experiments

September 11, 2026

Abstract

We replace the fixed B-spline basis of SplineCNN by learnable rational (safe-Padé) basis functions. Because rational functions need not be localised, the number of basis functions K — and with it the dominant kernel-weight parameter count $K C_{\text{in}} C_{\text{out}}$ — decouples from the spatial resolution k^D of a spline grid. With a spline-subspace initialization and a variance-preserving weight rescaling, $K=4-8$ non-separable rational basis functions match or exceed $K=25-125$ B-spline hats on PascalVOC keypoint matching (DGMC) and FAUST shape correspondence, at $2-5\times$ fewer parameters. All numbers are from branch `worktree-spline-improve`; every table is reproduced by `python benchmarks/analyze_results.py`.

1 Motivation

SplineCNN [1] convolves over irregular graphs with a continuous, matrix-valued kernel expanded in a fixed B-spline basis. Because the hat functions are *localised*, covering the pseudo-coordinate domain $[0, 1]^D$ at resolution k requires $K = k^D$ basis functions, each carrying its own weight matrix: $K=25$ in 2-D and $K=125$ in 3-D for $k=5$. The kernel weights $K C_{\text{in}} C_{\text{out}}$ dominate the parameter count of every SplineCNN we study (e.g. 95% of a DGMC backbone).

We replace the hats by *learnable rational basis functions* (safe Padé approximants, as in PAU [3], rKAN [4] and KAT [5], which use them as *activations*; here they are *spatial kernel bases*). Rational functions are smooth, bounded by construction, and — crucially — need not be localised: the number of basis functions K becomes a free capacity parameter decoupled from any spatial resolution. Our main empirical finding is that $K=4-8$ non-localised rational functions match or beat $K=25-125$ hats.

2 The operator

All layers compute the SplineConv form

$$\mathbf{x}'_i = \Theta_{\text{root}} \mathbf{x}_i + \square_{j \in \mathcal{N}(i)} \left(\sum_{p=1}^K B_p(\mathbf{u}_{ij}) \Theta_p \right) \mathbf{x}_j + \mathbf{b}, \quad (1)$$

with pseudo-coordinates $\mathbf{u}_{ij} \in [0, 1]^D$ (normalised Cartesian offsets, `T.Cartesian`), weight matrices $\Theta_p \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$, and aggregation $\square \in \{\text{mean}, \text{add}\}$ (mean unless stated; the FAUST reference protocol uses add, see §6). Only the basis $\{B_p\}$ differs between models. Throughout,

$t = 2u - 1 \in [-1, 1]$ and $\phi_0, \phi_1, \dots$ are Chebyshev polynomials of the first kind ($\phi_0=1$, $\phi_1=t$, $\phi_{j+1}=2t\phi_j - \phi_{j-1}$; `--poly monomial` is supported but slightly worse).

B-spline baseline (`dgm/models/bspline.py`, pure PyTorch). B_p are tensor products of degree-1 open B-splines (“hats”) on k uniform knots per dimension; 2^D of the $K = k^D$ products are non-zero at any $\mathbf{u}$.

Product rational basis (`RationalBasis`, `--rational_basis product`). $B_p(\mathbf{u}) = \prod_{d=1}^D r_{p_d}^{(d)}(u_d)$, a tensor product of per-dimension learnable rational functions

$$r_k(u) = \frac{P_k(t)}{1 + |Q_k(t)|}, \quad P_k(t) = \sum_{j=0}^m a_{k,j} \phi_j(t), \quad Q_k(t) = \sum_{j=1}^n b_{k,j} \phi_j(t), \quad (2)$$

with degrees $(m, n) = (5, 4)$ (`--degrees 5 4`). This is the *safe* Padé form, variant B; variant A uses $1 + \sum_j |b_{k,j} \phi_j(t)|$ (`--safe A`, slightly worse). The denominator is ≥ 1 , so no pole can form. Indexing matches the hat basis (first dimension fastest), so the Θ_p are shape-compatible with SplineConv. Still $K = \prod_d k_d$ functions.

Multivariate rational basis. Each basis function is a single *non-separable* rational function of all coordinates (`MultivariateRationalBasis`, `--rational_basis multivariate`),

$$B_p(\mathbf{u}) = \frac{P_p(\mathbf{t})}{1 + |Q_p(\mathbf{t})|}, \quad P_p(\mathbf{t}) = \sum_{|\alpha| \leq m} a_{p,\alpha} \prod_d \phi_{\alpha_d}(t_d), \quad Q_p(\mathbf{t}) = \sum_{1 \leq |\alpha| \leq n} b_{p,\alpha} \prod_d \phi_{\alpha_d}(t_d), \quad (3)$$

with multi-indices $\alpha \in \mathbb{N}_0^D$ of *total degree* $|\alpha| \leq m$ (numerator, $\binom{m+D}{D}$ terms) and $|\alpha| \leq n$ (denominator, $\binom{n+D}{D} - 1$ terms, no constant). We use $(m, n) = (8, 6)$ — a product of two degree-5 polynomials has total degree 10, so $(5, 4)$ underfits the hat targets (fit RMSE $3\times$ worse) and yields no gain over SplineCNN. K is **free** (`--num_bases K`), independent of any grid. Coefficient counts are negligible: at $D=2$, $(8, 6)$ has $45+27 = 72$ scalars per basis function.

MLP control (`MLPBasis`, `--rational_basis mlp`). $\mathbf{u} \mapsto \text{Lin}_K(\text{ReLU}(\text{Lin}_{64}(\mathbf{t})))$ — the filter-generating network of SchNet/PointConv with the same Θ_p , used to attribute gains to the rational *form* rather than to “any learned K -dimensional basis”.

Cost. Parameters: $K C_{\text{in}} C_{\text{out}} + C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ per layer plus the basis coefficients. Compute: rational bases evaluate all K functions densely per edge (no sparsity), costing $\sim 12\text{--}40\%$ more time per epoch than the pure-PyTorch spline at equal K in 2-D and $\sim 2\times$ at large feature counts; decreasing K recovers most of it.

3 Initialization

Initialization is the single most important design choice; a bad one costs 0.7–90 accuracy points (§7).

Spline (hat) fit — default for grid- K . Fit each rational function to its B-spline hat by least squares so the layer *starts as a SplineConv*: closed-form LS fit of the numerator with $Q \equiv 0$, then joint refinement of (a, b) with Adam (lr 0.02, 500 steps) followed by L-BFGS (100 iterations, strong-Wolfe line search), in float64, on a fixed grid (1-D: 256/1024 points; 2-D: 32^2 ; 3-D: 16^3). Two essential details:

1. **Never start the denominator at exactly zero.** $\partial|Q|/\partial b = \text{sign}(Q) \phi_j$ vanishes identically at $b = 0$ ($\text{sign}(0) = 0$), freezing the denominator forever — the model silently degrades to a polynomial basis. We start $b \sim \mathcal{N}(0, 0.01^2)$.
2. The fit is cached per configuration within a process, so stacked layers share one fit (~ 20 s in 3-D instead of ~ 45 min for a 6-layer model).

PCA-of-hats — for free K . When $K \neq k^D$, fit the K functions to the K leading principal components (left singular vectors, deterministic sign, rescaled to unit max) of the k^D hat basis evaluated on the grid: the K -dimensional subspace that best approximates the SplineConv kernel space. $K = k^D$ recovers the hats exactly. Caveat: components come in degenerate symmetric pairs, and the higher components are oscillatory — at $K \geq 16$ in 3-D this init is seed-fragile (§6).

Variance-preserving (vp) gain (adapted from KAT [5]). After basis init, scale the kernel weights by $\sqrt{\alpha}$ with

$$\alpha = \mathbb{E}_{\mathbf{u}} \left[\sum_p \text{hat}_p(\mathbf{u})^2 \right] / \mathbb{E}_{\mathbf{u}} \left[\sum_p B_p(\mathbf{u})^2 \right] \quad (4)$$

(numerical integration on a grid), so any init produces messages at SplineConv scale. $\alpha \approx 1.00$ for hat-fitted bases (a no-op), 0.57 for Gaussian-fitted, 0.06–0.14 for raw-Chebyshev, 0.018 for constant-one. It rescues mis-scaled inits (raw-Chebyshev on WILLOW: 46% $\rightarrow$ 97%) but never substitutes for a good target shape.

What fails, and why (ablation summary). Ranking over PascalPF / WILLOW / PascalVOC: hat-fit $\gtrsim$ Gaussian-bump fit $\gg$ raw Chebyshev (+vp) $\gg$ random ($B_p \approx 1/K$; -0.7 to -5.8 pts) $\gg$ constant-one ($B_p \equiv 1$, catastrophic). The constant init fails for two reasons: (i) without vp its output scale is $\sqrt{K}$ -fold too large and the DGMC softmax saturates (gradients $\rightarrow 0$); (ii) *with* vp it still fails, because all K identical basis functions give all Θ_p identical gradients, and Adam’s per-parameter normalisation then moves $\sum_p \Theta_p$ $K \times$ faster than the learning rate suggests — verified: training survives at lr/ K but not at lr/5. Weight decay on basis coefficients only (`--basis_wd`) had no measurable effect (-0.26 , n.s.).

4 Training details (common)

Adam (PyTorch defaults, $\beta = (0.9, 0.999)$), no weight decay. The basis coefficients are trained jointly with everything else at the same learning rate. Kernel weights use SplineConv’s uniform init with bound $1/\sqrt{K C_{\text{in}}}$ (matching PyG’s root/bias init instead — bound $1/\sqrt{C_{\text{in}}}$, zero bias — was tested and changes nothing: 97.31 vs 97.23 on FAUST). All experiments log to wandb (losses split into the $S^{(0)}/S^{(L)}$ terms where applicable, train/test accuracy, per-step gradient norm, update norm $\|\Delta\theta\|$, parameter norm, timings).

5 Experiment 1 — PascalVOC keypoint matching (DGMC)

Setup (`examples/pascal.py`, defaults unless noted). DGMC [2] on PascalVOC-Keypoints with Berkeley annotations: graphs from `T.Delaunay + FaceToEdge` (tolerant variant: < 3 -keypoint graphs have no faces), node features = frozen VGG16-bn `relu4_2 || relu5_1` (1024-d), pseudo-coordinates 2-D anisotropic `T.Cartesian` ($D=2$). Backbones ψ_1 ($1024 \rightarrow 256$, `cat=False`, dropout 0.5) and ψ_2 ($128 \rightarrow 128$, `cat=True`, dropout 0), both 2 conv layers; $L=10$ consensus steps. Adam lr 10^{-3} , batch 512 graph pairs, **15 epochs**, self-supervised pairs (`ValidPairDataset`), test: 1000 samples/category, metric = mean over the 20 categories of keypoint-matching accuracy at the final epoch; **5 seeds** (0–4). Rational configs: multivariate, $(m, n) = (8, 6)$, safe B, Chebyshev; PCA init + vp for free- K ; hat-fit init for $K = k^D$ (“mv $k=3$ ” = `kernel_size 3`, $K=9$).

config	K	params	final acc	best acc
SplineCNN $k=5$	25	9.50M	72.00 ± 0.39	73.00 ± 0.40
SplineCNN $k=3$	9	3.74M	72.06 ± 0.85	72.78 ± 0.40
SplineCNN $k=2$	4	1.93M	70.94 ± 0.63	71.48 ± 0.40
product rational $k=5$	25	9.50M	72.24 ± 0.67	72.88 ± 0.48
product rational $k=3$	9	3.74M	72.62 ± 0.41	73.26 ± 0.18
multivariate, hat init, $k=3$	9	3.74M	73.76 ± 0.81	74.12 ± 0.48
multivariate, PCA+vp, $K=4$	4	1.94M	72.92 ± 0.36	73.78 ± 0.41
multivariate, PCA+vp, $K=6$	6	2.66M	73.10 ± 0.41	74.06 ± 0.37
multivariate, PCA+vp, $K=9$	9	3.74M	73.16 ± 0.47	73.72 ± 0.19
multivariate, PCA+vp, $K=16$	16	6.26M	72.26 ± 0.42	73.26 ± 0.42
multivariate, PCA+vp, $K=2 / K=1$	2 / 1	1.21M / 0.85M	$71.44 / 67.98$	
MLP basis, $K=4 / K=9$	4 / 9	1.94M / 3.74M	$72.12 \pm 0.32 / 71.46 \pm 0.64$	
multivariate (5, 4), $k=5$ (degree control)	25	9.51M	72.04 ± 0.94	

Table 1: PascalVOC keypoint matching (DGMC), mean $\pm$ std over 5 seeds.

Welch t-tests: mv-hat $k=3$ vs SplineCNN $k=5$: $+1.76$, $p = 0.005$; vs product $k=3$: $+1.14$, $p = 0.032$. $K=4$ vs SplineCNN $k=5$: $+0.92$, $p = 0.005$; vs MLP $K=4$: $+0.80$, $p = 0.006$; vs SplineCNN $k=2$ (equal K and params): $+1.98$, $p = 0.001$. The accuracy- K curve is flat for $K \in [4, 9]$ and *drops* at $K=16$ (overfitting); the (5, 4) control shows the total degree matters.

6 Experiment 2 — FAUST shape correspondence

Setup (`examples/faust.py`, mirroring the SplineCNN paper / PyG example). 100 registered human meshes (6890 vertices, 13776 faces), 80 train / 20 test; task: classify every vertex of a test mesh into one of 6890 template-vertex classes; metric: exact-match accuracy. Input `T.Constant(1)`; pseudo-coordinates 3-D `T.Cartesian` ($D=3$). Network: 6 conv layers ($1 \rightarrow 32 \rightarrow 64 \times 5$), ELU, then `Lin(64, 256)`, ELU, dropout 0.5, `Lin(256, 6890)`, log-softmax; NLL loss. Adam lr $0.01 \rightarrow 0.001$ at epoch 61, batch 1, **100 epochs**, **gradient-norm clipping 1.0** (all backbones — see below), aggregation `add` (the PyG reference) unless “mean”; **3 seeds**. Rational configs: multivariate (8, 6), PCA init + vp, `--num_bases K`; product $k=3$ with hat init.

Notes. (i) *Reproduction gap and stability*: the PyG example’s `aggr='add'` deviates from the paper’s degree-normalised operator; with mesh degree ≈ 6 it destabilises training (2/3 seeds collapse to uniform output after a gradient spike) and costs ≈ 1.5 pts even when rescued by

config	K	params	final acc
multivariate $K=8$	8	1.97M	99.41 $\pm$ 0.04
multivariate $K=4$	4	1.89M	99.38 $\pm$ 0.09
product rational $k=3$	27	2.30M	99.24 $\pm$ 0.08
<i>SplineCNN paper (CUDA kernel)</i>	125	4.1M	99.20
multivariate $K=4$, mean aggr	4	1.89M	98.93 $\pm$ 0.10
SplineCNN $k=5$, mean + clip (best reproduction)	125	4.11M	98.71 $\pm$ 0.08
MLP basis $K=8$	8	1.96M	98.21 $\pm$ 1.11
SplineCNN $k=5$, add + clip	125	4.11M	97.23 $\pm$ 0.51
SplineCNN $k=3$	27	2.30M	83.6 $\pm$ 15.6
multivariate $K=16$ / $K=27$	16 / 27	2.13M / 2.34M	70.4 $\pm$ 25.2 / 19.0 $\pm$ 32.8
SplineCNN $k=5$, add, no clip (literal PyG example)	125	4.11M	29.1 $\pm$ 50.3
SplineCNN $k=2$	8	1.95M	3.4 $\pm$ 3.3

Table 2: FAUST shape correspondence, mean $\pm$ std over 3 seeds.

clipping. With mean aggregation + clipping, SplineCNN reproduces to within 0.5 pts of the paper. Clipping is required under either aggregation (mean/no-clip: 96.9 $\pm$ 1.7). (ii) *Small K* : SplineCNN cannot be made small in 3-D ($K=27$ loses a seed, $K=8$ fails entirely), whereas $K=4$ –8 rational functions exceed the published SplineCNN number at $2.2\times$ fewer parameters and the lowest variance in the table — under the *harder* add protocol. (iii) *Large- K fragility*: with PCA init, $K \geq 16$ in 3-D is seed-fragile (slow or collapsed in 2/3 seeds) — an init problem (oscillatory components, small vp gain), not a capacity limit. (iv) The MLP control shows non-localisation alone helps, but the rational form adds +1.2 pts and $25\times$ lower variance.

7 Secondary experiments (summary)

WILLOW-ObjectClass (DGMC, no VOC pretraining, 10 runs $\times$ 15 epochs, 20 train images/class): spline 96.80 $\pm$ 1.20, product rational 97.28 $\pm$ 1.01, multivariate $k=5$ 97.55 $\pm$ 0.63 (all n.s.; duck dominates variance). Per-epoch curves: all bases memorise the 380 train pairs/class by epoch ~ 8 ; test accuracy saturates without decay, test loss drifts up mildly (over-confidence). Init ablation (product basis): Gaussian-fit 97.72 (best), hat-fit 97.28, cheb+vp 97.07, random+vp 92.13, constant (any variant) 10.00 = chance.

PascalPF (synthetic + PascalPF pairs, 3 seeds $\times$ 20 epochs): rational $k=3$ 95.73 $\pm$ 0.06 > spline $k=5$ 95.27 $\pm$ 0.15; Gaussian-fit init 95.90; random init 94.93; constant init 78/66. On PascalVOC the same init ordering holds but reversed between gauss and hat: hat-fit 73.76 > gauss 72.40 ($p = 0.018$) > cheb+vp 71.62.

8 Practical recipe

1. Use the multivariate basis, total degrees (8, 6), safe-B, Chebyshev.
2. Choose $K \approx 4$ –8, independent of dimension.
3. Init: hat-fit when $K = k^D$, else PCA-of-hats; always with the vp gain; never a zero denominator.

4. Train exactly like the SplineCNN you are replacing; add grad-norm clipping for deep stacks with add-aggregation.
5. Expect $\approx$ equal accuracy at $\sim 2\text{--}5\times$ fewer parameters, or $+0.7\text{--}1.8$ pts at equal parameter count; expect 10–40% slower epochs.

9 Open questions / limitations

- Large- K PCA-init fragility in 3-D (localised targets for the higher components should fix it).
- Basis-coefficient scale drift at large K (safe-Pad  gauge freedom; benign on accuracy, visible in parameter norms / calibration).
- Mean vs add aggregation interacts with the basis (mean helps spline, mildly hurts rational on FAUST); unexplained.
- Timing uses a pure-PyTorch SplineConv baseline; against the CUDA `torch-spline-conv` kernel the rational overhead would be larger.
- WILLOW/PascalPF gains are within seed noise; the significant results are PascalVOC and FAUST.

References

- [1] M. Fey, J. E. Lenssen, F. Weichert, and H. M ller. SplineCNN: Fast geometric deep learning with continuous B-spline kernels. In *CVPR*, 2018.
- [2] M. Fey, J. E. Lenssen, C. Morris, J. Masci, and N. M. Kriege. Deep graph matching consensus. In *ICLR*, 2020.
- [3] A. Molina, P. Schramowski, and K. Kersting. Pad  activation units: End-to-end learning of flexible activation functions in deep networks. In *ICLR*, 2020.
- [4] A. A. Aghaei. rKAN: Rational Kolmogorov–Arnold networks. *arXiv:2406.14495*, 2024.
- [5] X. Yang and X. Wang. Kolmogorov–Arnold transformer. In *ICLR*, 2025.